

Penny-Wise, Cache-Foolish: When Token-Optimal MCP Tool Retrieval Isn't Cost-Optimal

[Preprint v1.2 — 2026-10-03]

Anoshor B. Paul* Chirag Pradhan Krishn Maloo

Independent Researchers

anoshorpaul@gmail.com

Abstract

Agents built on the Model Context Protocol (MCP) increasingly face catalogs of hundreds of tools, and a growing literature proposes retrieving only the relevant tool schemas to shrink the prompt. These methods report their efficiency as raw token reduction. Commercial LLM APIs, however, bill cached prompt prefixes at roughly one tenth of the uncached price and may charge a premium to write them, so the token count a method minimizes is not the quantity a deployment pays for. We analyze this gap with a closed-form cost model of tool exposure under prefix caching and a trace-driven counterfactual replay of 95 real LiveMCPBench trajectories (525 MCP tools, 119k schema tokens) under four verified provider pricing regimes. For a single short task, retrieval is both token- and cost-optimal, costing $0.09\times$ as much as loading every tool with caching. As sessions grow, however, the ranking inverts. In ten-task sessions over a 100-tool catalog, per-step retrieval sends 42% fewer tokens yet costs $1.60\times$ more (cluster-valid 95% CI 1.45–1.74 over disjoint sessions), and is more expensive in every session we replay. Our pre-registered hypothesis that retrieval-in-prefix costs more than static caching for sessions of ≥ 20 calls and ≤ 150 tools holds under all three tested regimes ($1.25\text{--}1.37\times$). We derive a break-even rule (retrieval pays only when the omitted catalog exceeds $\frac{W-R}{2R} \bar{h}(m-1)$ tokens) that predicts the replay’s break-even catalog size within 7.8% mean error under Anthropic/OpenAI pricing. Two design choices matter more than the retrieval algorithm. Placing retrieved schemas after the conversation history is the cheapest strategy in every ten-task configuration of the replay grid, though the one cell we could bill puts it level with per-request retrieval. Non-deterministic schema serialization alone inflates static-caching cost $1.7\text{--}4.8\times$. A live billing check on **gpt-5-nano** (1,151 requests) puts the replay within 17.4% of billed cost and shows that it understates the real cost of retrieval, so the reported crossover is conservative. Tool-selection accuracy meanwhile falls from 71% to 41% as the exposed catalog grows from 16 to 128 tools, so neither exposing everything nor re-selecting per turn is free; pricing those errors shrinks the inversion region without eliminating it.

Keywords: Model Context Protocol, tool retrieval, prompt caching, LLM agents, inference cost, context engineering

1 Introduction

The Model Context Protocol (MCP) standardizes how LLM applications discover and invoke external tools [1]. Its success has created a scaling problem: an agent connected to a handful of MCP

*Corresponding author.

servers can see hundreds of tools, and clients that expose every schema to the model repeatedly send a large tool block. A research response is tool retrieval: select the few tools relevant to the current request and expose only their schemas. RAG-MCP reports a prompt-token reduction of over 50% [9], MCP-Zero a 98% reduction on APiBank [10], vector-based discovery 99.6% [11], and Tool Attention a 95% reduction in tool tokens per turn [12].

These numbers share a unit, the raw input token, that is no longer what commercial APIs bill. Anthropic, OpenAI and Google all cache prompt prefixes. Tokens that match an earlier request are billed at about $0.1\times$ the base input price, while writing a new prefix costs $1.0\text{--}2.0\times$ depending on provider and time-to-live [22, 24, 27]. Tool definitions sit at the very start of that prefix. Changing which tools are exposed therefore invalidates not only the tool block but everything after it, including a conversation history that may be much larger than the tools themselves [22]. A method that minimizes tokens by changing the tool set can, in principle, raise the bill.

When this cost inversion occurs has not been measured on a common set of tool-use traces. Lumer et al. evaluate prompt caching on long-horizon agents and advise against dynamic function calling, but test no retrieval method and give no cost model [13]. Tool Attention places retrieved schemas late in the prompt to protect the cache, but its cost figures are projections rather than billed measurements [12]. TokenPilot relocates tool definitions downstream and measures real cache hits on one model, without comparing MCP retrieval strategies [14]. Provider-native tool search (`defer_loading`) injects discovered tools at the end of the context expressly to preserve the cache [23, 25], but has not been independently compared with the academic retrievers.

This paper asks: under current prefix-cache pricing, when is token-optimal MCP tool retrieval also cost-optimal, and when is it not? The scope is deliberately narrow: one single-agent benchmark, API-billed models, and cost measured at the billing interface rather than inside a serving stack. We make five contributions:

1. A closed-form cost model of tool exposure under prefix caching, with a break-even rule for when retrieval pays off (Section 3).
2. A trace-driven counterfactual replay that bills nine exposure strategies over 95 real LiveMCPBench trajectories, 525 real MCP tool schemas and four verified pricing regimes, at zero API cost (Section 4).
3. Evidence that the token and cost rankings agree for single short tasks but invert in multi-task sessions, confirming a pre-registered hypothesis, and that the break-even rule predicts where the inversion begins (Section 5).
4. Practical guidance: cache-aware placement of retrieved schemas matters more for cost than the choice of retriever (Section 6).
5. Evidence that a mechanical defect, non-deterministic serialization of tool definitions, can cost more than every strategy choice in this paper combined, and is free to fix (Section 5.5).

2 Background and Related Work

MCP and tool exposure. MCP specifies discovery through `tools/list` and execution through `tools/call`, but leaves model-facing presentation to the client [2]. An MCP client may aggregate tools from several servers and present them as function definitions in a request’s `tools` field. The 2026-07-28 specification states that servers “SHOULD return tools in a deterministic order” because this “improves LLM prompt cache hit rates when tools are included in model context” [2], which

is the server-side counterpart of the client-side defect we measure in Section 5.5. Benchmarks such as LiveMCPBench (95 tasks, 70 servers, 527 tools) [16], MCP-Bench [17], MCP-Universe [18] and MCP-AgentBench [19] evaluate agents on real servers; they report success and, at most, raw token usage. Hasan et al. show that most MCP tool descriptions contain quality defects that affect selection [20], which bears on how large each schema is.

Tool retrieval. RAG-MCP retrieves relevant tool descriptions from an external index before calling the model [9]. MCP-Zero lets the agent actively request tools as it works [10]. Semantic tool discovery embeds tools and retrieves the top- k per query [11]. Tool Attention keeps compact summaries of all tools in the prompt and loads full schemas only for gated tools, placing them just before the latest user message; its 95% tool-token reduction comes from a simulated benchmark, while end-to-end cost is projected [12]. Code-execution designs expose tools as a programmatic API instead of function definitions [21]; we leave them to future work.

Prompt caching. All three major providers cache prompt prefixes. Anthropic bills cache writes at $1.25\times$ (5-minute TTL) or $2\times$ (1-hour TTL) and reads at $0.1\times$. Its prefix hierarchy runs tools $\rightarrow$ system $\rightarrow$ messages, and a change at any level invalidates all later levels [22]. OpenAI caches automatically. On GPT-5.6 and later, writes cost $1.25\times$ and reads $0.1\times$, while earlier models have no write premium; any change to tool names, descriptions, schemas or ordering prevents reuse [24]. Gemini’s implicit cache bills cached tokens at $0.1\times$ (e.g., \$0.075 vs. \$0.75 per million on Gemini 3.8 Flash) with a 4,096-token minimum [27]. Lumer et al. measure 41–80% cost savings from caching across providers [13]. TokenPilot reconciles context compression with cache alignment [14]. Serving stacks reuse prefixes internally through RadixAttention and automatic prefix caching [3, 4], and ReCache makes KV blocks composition-invariant for self-hosted serving [15]. None of this helps a developer who buys inference through an API: the reuse they can influence is the one the provider bills, which is what we model. A public gateway issue documents tool-schema property order changing between syncs in 44% of consecutive request pairs, silently breaking the cache [28].

Gap. No prior work gives a cost model for tool-exposure strategies under prefix caching, compares the retrieval families on the same real traces with cache-aware billing, or tests whether the token ranking reported by retrieval papers survives caching.

3 A Cost Model of Tool Exposure

Billing. Normalize the uncached input price to 1. Suppose a call’s prompt has x tokens in total, of which the first c match a prefix already in the cache. Its input costs $Rc + W(x - c)$, where R is the read multiplier and W the write multiplier. Output tokens cost O each. Cached tokens count only when the shared prefix reaches the provider’s minimum length.

Prompt layout. At call t the prompt is a tool block B_t , a system prompt S , and an append-only history H_t . Strategies differ only in what B_t contains and where retrieved schemas sit (Table 1). Let C be the tokens of the full catalog, b the tokens of a retrieved subset, $\bar{h}$ the mean tokens appended per call, T the number of calls and m the number of user requests (tasks) in the session.

Break-even for per-request retrieval. A static cached catalog pays for C once at the write price and then at the read price on each call. Retrieval that changes the tool set at each new request saves $(C - b)$ tokens on every call, worth about $(C - b)[W + R(T - 1)]$ over the session. But each

Table 1: Tool-exposure strategies replayed. “Prefix” means the retrieved schemas occupy the tool block at the start of the prompt.

ID	Strategy	Represents
S0	All schemas in prefix, caching disabled	No-cache baseline
S1	All schemas in prefix, cached	Naive production default
S1u	S1 with tool JSON re-serialized non-deterministically ($p=0.44/\text{call}$)	Gateway bug [28]
S2	Retrieved schemas in prefix, re-retrieved per step	MCP-Zero-style active discovery [10]
S2r	Retrieved schemas in prefix, retrieved per request	RAG-MCP-style [9]
S2u	Prefix holds the union of everything retrieved so far	“Sticky” fix
S3	Summaries of all tools in prefix; retrieved schemas after history	Tool Attention layout [12]
S4	Two meta-tools; discovery results appended to history	Recorded agent; Anthropic tool search [23]
S5	Names+descriptions in prefix; schemas appended on discovery	OpenAI tool search [25]

change invalidates the history behind it: at the j -th request, the history of roughly $(j-1)\bar{h}T/m$ tokens is written at W instead of read at R . Summing over the $m-1$ switches gives an extra cost of $(W-R)\bar{h}T(m-1)/2$. Retrieval is cheaper if and only if

$$C - b > \frac{(W-R)\bar{h}T(m-1)}{2(RT+W-R)} \xrightarrow{T \gg 1} \frac{W-R}{2R}\bar{h}(m-1). \quad (1)$$

Under Anthropic’s 5-minute pricing or OpenAI GPT-5.6+ ($W=1.25$, $R=0.1$) the constant $\frac{W-R}{2R}$ is 5.75; without a write premium ($W=1$) it is 4.5. In words, retrieval pays only if the tools you leave out are worth more tokens than about six turns’ worth of history growth, per re-retrieval. The saving grows with the catalog, but the penalty grows with the history, and history grows with session length.

Per-step retrieval. If the tool set changes on a fraction q of calls rather than once per request, the same argument gives a threshold of about $\frac{W-R}{2R}q\bar{h}T$, which grows with T itself.

Placement. Equation 1 assumes retrieved schemas sit in the prefix. If instead they are placed after the history (tail placement) or appended when discovered (append-only), a change in the tool set leaves the history’s cache intact, and the cache-break term disappears. The price is that tail schemas are re-sent uncached on every call, and appended schemas accumulate.

4 Methodology

Data. We use all 95 trajectories released with LiveMCPBench [16]. They were produced by a Claude Sonnet 4 agent that discovers tools through a `route` meta-tool backed by Qwen3-Embedding retrieval and invokes them through an `execute-tool` meta-tool. Each trajectory records every assistant message, tool call and tool result, and carries a judge-assigned success label (0.74 success rate under the Claude Sonnet 4 judge). We tokenize all messages and the benchmark’s 525 parseable tool schemas with the `o200k_base` tokenizer. The catalog totals 119,374 schema tokens (mean 227, median 137, maximum 2,044 per tool). Trajectories average 9.5 LLM calls (6.5 excluding pure discovery calls), and each non-discovery call appends $\bar{h} = 949$ tokens on average. All 222 distinct tools returned by retrieval match catalog entries.

Counterfactual replay. Cost depends only on the shape of the token flow (which segments appear, in what order, at each call) and on provider billing rules. For each trajectory and strategy we reconstruct the ordered segments of every prompt the strategy would send, holding the agent’s actions fixed. Strategies without a discovery tool (S0–S3) drop the route-only calls and their results, since those calls would not exist.

- For S2, the retrieved set at each call is the most recent route result.
- For S2r, it is the union of the task’s route results, fixed for the duration of the request (an optimistic retriever).
- For S2u, it is the union of everything retrieved so far in the session.

Each call is billed by the rule of Section 3, with the cached prefix computed as the longest segment-exact prefix shared with an earlier prompt in the session. Because history is append-only, the most recent earlier prompt with the same tool block always gives the longest match, so replay is linear in session length. Output is billed at $O=5$.

Sessions and catalogs. Real assistants serve several requests per conversation. We form 95 sessions of $m \in \{1, 2, 3, 5, 10\}$ tasks by chaining trajectories in a fixed seeded order, so that each task starts exactly one session. A new request appends the user’s question to the history and triggers per-request retrieval. The catalog for a session of size $N \in \{25, 50, 100, 200, 525\}$ always contains every tool the session retrieves or executes, filled to N with a seeded random sample of the remaining tools.

Pricing regimes. All four were verified against provider documentation on 2026-09-19:

- P1: $W=1.25$, $R=0.1$, minimum 1,024 tokens (Anthropic 5-minute TTL; OpenAI GPT-5.6+) [22, 24].
- P2: $W=1$, $R=0.1$, minimum 1,024 (OpenAI GPT-5.5 and earlier) [24].
- P3: $W=2$, $R=0.1$, minimum 1,024 (Anthropic 1-hour TTL) [22].
- P4: $W=1$, $R=0.1$, minimum 4,096 (Gemini implicit caching) [27].

Statistics and hypotheses. We report per-session cost ratios against S1 with 95% percentile-bootstrap confidence intervals (4,000 resamples, paired by session). Before any replay we registered four hypotheses in the project’s Phase-1 scoping record:

- H1: for $N \leq 150$ and $T \geq 20$ calls, retrieval-in-prefix costs more than static caching under at least two of three regimes.
- H2: the analytical model predicts billed cost within 20% MAPE.
- H3: cache-aware placement is on the cost–accuracy Pareto frontier for $N \geq 100$.
- H4: static exposure loses selection accuracy as N grows.

Accuracy proxy. Replay cannot measure how exposure changes agent behavior. As a zero-cost proxy we compute availability: the share of executed tools that were inside each retrieval-in-prefix strategy’s tool set at the moment of execution. A tool outside the set could not have been called.

Table 2: Single-task sessions, $N=525$, pricing P1. Cost is the mean per-task ratio to S1 with 95% bootstrap CI.

Strategy	Raw input tokens/task	Cost vs. S1	95% CI	Cache hit
S0 static, no cache	807,818	3.10	[2.85, 3.37]	0%
S1 static, cached	807,818	1.00	—	75%
S1u static, unstable JSON	807,818	2.29	[2.10, 2.48]	41%
S2 RAG-prefix, per step	31,495	0.09	[0.08, 0.10]	46%
S2r RAG-prefix, per request	41,377	0.09	[0.08, 0.10]	55%
S2u RAG-prefix, sticky union	38,950	0.10	[0.08, 0.11]	50%
S3 summaries + tail schemas	103,634	0.18	[0.17, 0.19]	68%
S4 discovery, append-only	54,354	0.10	[0.09, 0.11]	60%
S5 name+desc + append	312,138	0.34	[0.33, 0.35]	82%

Live validation. A harness replays each strategy’s exact prompt sequence against `gpt-5-nano` (\$0.05 per million uncached and \$0.005 per million cached input tokens [26]) with a 64-token output allowance. It reads the billed `cached_tokens` to test H2 against real billing, measures tool-selection accuracy with $N \in \{16, 32, 64, 128\}$ candidate tools (H4), and probes the reported 128-tool request limit [29]. The whole run is guarded by a hard \$1.50 cap; Section 5.8 reports the outcome.

5 Results

5.1 Single tasks: token-optimal is cost-optimal

Table 2 shows the replay for one task per session with the full 525-tool catalog under P1. Every retrieval variant costs about a tenth of static caching (S2r $0.09\times$, 95% CI 0.08–0.10), and the token and cost rankings largely agree. Measured churn is low: once discovery calls are removed, the retrieved set changes between consecutive calls in only 7% of cases on average (median 0%). The agent retrieves early and then executes, so a prefix-resident tool set is rarely invalidated within a task. Results under P2–P4 are qualitatively identical. P4’s 4,096-token minimum lowers cache hits for the small retrieved prefixes (S2 hit rate 22% vs. 46%), but S2 still costs only $0.11\times$.

5.2 Sessions: the ranking inverts

Figure 1 maps the cost of per-request retrieval (S2r) relative to static caching across session length and catalog size. Retrieval’s advantage shrinks with every additional request in the session and grows with catalog size. With three tasks per session it already loses at $N=25$ ($1.12\times$). With ten tasks it loses up to $N=100$ ($1.31\times$) and breaks even at $N=200$ ($1.00\times$). Only the full 525-tool catalog keeps retrieval ahead at ten tasks ($0.56\times$).

Table 3 tests the pre-registered H1 over every session with at least 20 LLM calls and $N \in \{25, 50, 100, 150\}$. Per-step retrieval (S2) costs $1.25\text{--}1.37\times$ static caching under all three regimes, and every confidence interval excludes 1. Per-request retrieval (S2r) costs $1.07\text{--}1.15\times$. H1 is supported under three of three regimes, where two were required.

5.3 Fewer tokens, higher bill

Figure 2 plots raw input tokens against cost for $N=100$. For a single task the two axes agree: strategies that send fewer tokens cost less. For ten-task sessions they disagree. Per-step retrieval (S2) sends 2.08M tokens per session against static caching’s 3.57M (42% fewer), yet costs $1.60\times$ as

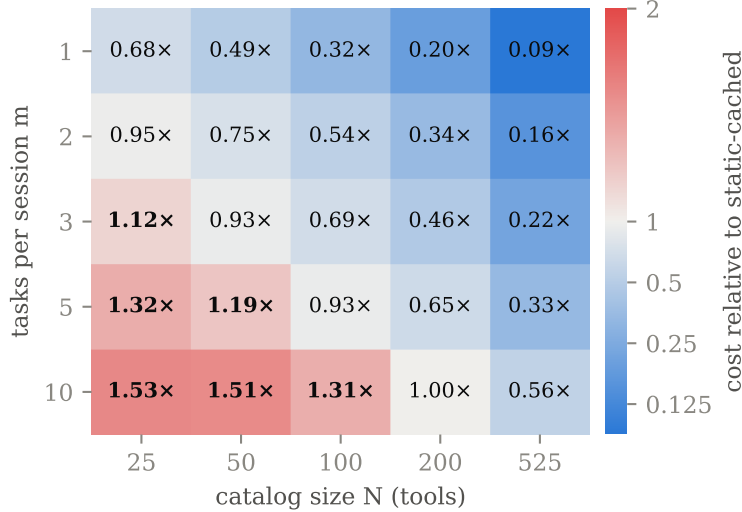


Figure 1: Cost of per-request retrieval-in-prefix (S2r) relative to static caching (S1), pricing P1. Blue: retrieval cheaper; red (bold): static caching cheaper. Each cell averages 95 sessions.

Table 3: H1: sessions with ≥ 20 LLM calls and $N \in \{25, 50, 100, 150\}$ ($n=880$ session-catalog pairs per row).

Regime	Strategy	Mean cost vs. S1	95% CI	Sessions costlier than S1
P1 ($W=1.25$)	S2 per step	1.37	[1.34, 1.40]	78%
	S2r per request	1.15	[1.13, 1.17]	68%
P2 ($W=1.00$)	S2 per step	1.25	[1.22, 1.27]	73%
	S2r per request	1.07	[1.05, 1.09]	60%
P4 (Gemini)	S2 per step	1.28	[1.25, 1.30]	76%
	S2r per request	1.10	[1.08, 1.12]	64%

much (95% CI 1.56–1.65) and is more expensive than S1 in all 95 sessions. The sticky-union “fix” (S2u) is the worst retrieval variant at $1.82\times$: every newly retrieved tool changes the prefix, and the prefix only grows.

5.4 Placement beats retrieval

Figure 3 compares all strategies at $N=100$. Tail placement (S3) is the most robust design. In ten-task sessions it is the cheapest strategy at every catalog size under all three regimes of the replay grid (Section 5.10 qualifies this against billed data), e.g., $0.94\times$, $0.82\times$ and $0.40\times$ static caching at $N=25$, 100 and 525 under P1. Its worst cell across the whole grid is $1.02\times$ under P1, $0.99\times$ under P2 and $1.13\times$ under P4, where Gemini’s 4,096-token minimum hurts it in short sessions over small catalogs. Append-only discovery (S4, the recorded agent’s own design) is next, with a worst cell of 1.24 – $1.27\times$, and it beats static caching in ten-task sessions only for larger catalogs. Per-request retrieval remains the best choice for single tasks, but the placement-aware designs trail it by 0.02–0.17 of S1’s cost.

5.5 Serialization stability can dominate every other choice

The 44% churn rate we use as input comes from a single production gateway report [28] and should be read as an illustrative rate, not an ecosystem constant; the mechanism, not the rate, is the

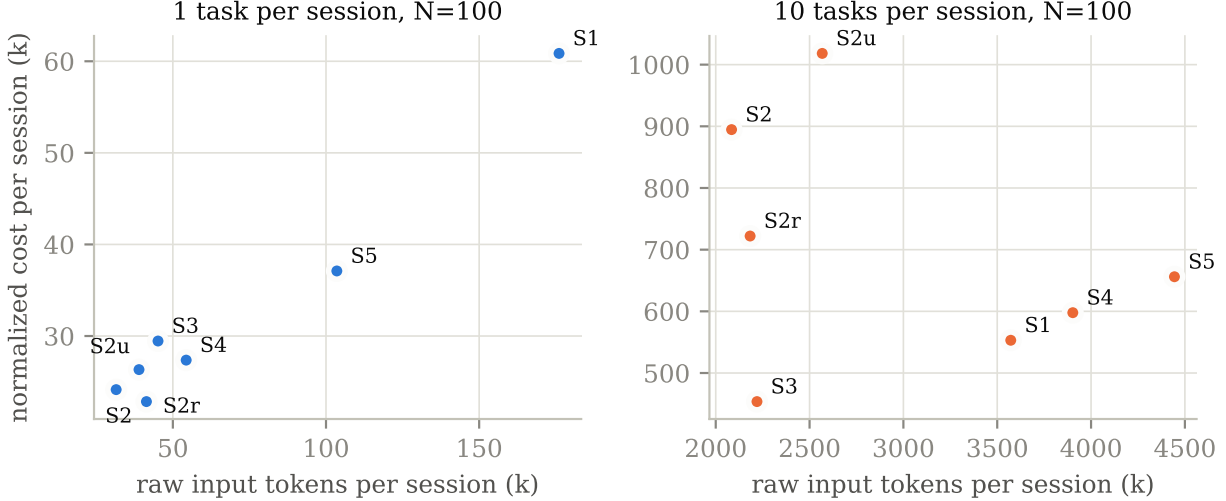


Figure 2: Raw input tokens (the metric retrieval papers report) versus normalized cost, $N=100$, P1. Left: single task. Right: ten-task sessions, where the lowest-token strategies (S2, S2r, S2u) are among the most expensive.

Table 4: Break-even catalog size N^* (tools): Equation 1 versus replay.

Regime	$m=2$		$m=3$		$m=5$		$m=10$	
	pred.	replay	pred.	replay	pred.	replay	pred.	replay
P1	19	20	36	40	76	84	189	198
P2	17	18	32	35	64	71	154	161
P4	17	34	32	47	64	80	154	165

finding. Randomly re-serializing the tool JSON on 44% of calls raises static-caching cost $1.7\text{--}4.8\times$ across the P1 grid (Figure 4). The penalty grows with session length and catalog size. In ten-task sessions with 100 tools this single defect costs $4.20\times$ (95% CI $4.10\text{--}4.31$), more than any choice between exposure strategies.

5.6 The break-even rule predicts the crossover

Table 4 compares the break-even catalog size predicted by Equation 1 with the size at which the replay’s mean S2r/S1 ratio crosses 1 (log-interpolated on a 17-point grid). The prediction uses the measured $\bar{h}=949$, 6.5 calls per task and a 1,680-token retrieved set. Under P1 and P2 the rule is within 4–11% in every cell (mean absolute percentage error 7.8%). Under P4 it underestimates short sessions (by 51% at $m=2$), because Gemini’s 4,096-token minimum prevents small prefixes from caching at all, which Equation 1 does not model; the error falls to 7% by $m=10$. The overall MAPE across the 12 cells is 14.4%. A first version of the rule, which omitted the one-time write term and averaged per-task ratios, overestimated break-even by about 40%; we report the corrected derivation.

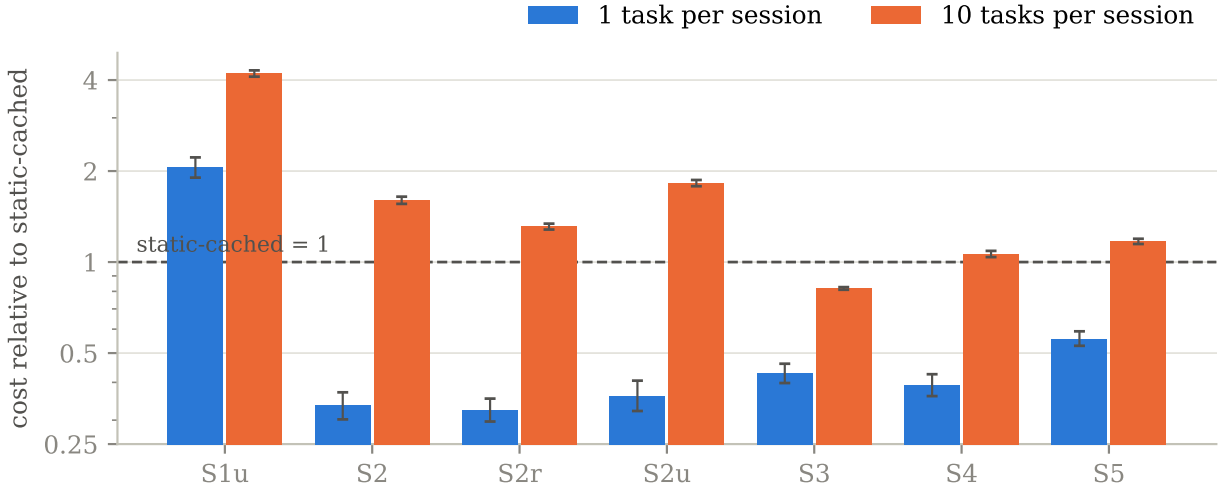


Figure 3: Cost relative to static caching (dashed line) at $N=100$, P1, for one-task (blue) and ten-task (orange) sessions. Error bars: 95% bootstrap CI. S1u = unstable serialization; S2/S2r/S2u = retrieval in prefix; S3 = summaries + tail schemas; S4/S5 = append-only discovery.

5.7 Accuracy proxy and sensitivity

Per-step retrieval would have hidden the tool the agent actually executed in 35% of calls (availability 64.8%, 342 of 528 executions). Per-request (99.2%) and sticky (98.5%) sets almost never miss, although S2r achieves this with an optimistic retriever that knows the task’s full tool needs. For per-step retrieval, cost and accuracy therefore point the same way: it is both the most cache-hostile and the most likely to hide the needed tool. We also randomly expired the cache on 10% or 30% of calls, as a proxy for TTL expiry during slow tool execution. This narrows the advantage of caching (S0 falls from $3.49\times$ to $1.74\times$ of S1 at 30%) but leaves the single-task ordering of strategies unchanged. Varying the output-to-input price ratio from $O=5$ to $O=0$ or $O=8$ does not change which strategy is cheapest in any tested cell. At $m=10$, $N=100$, per-step retrieval costs $1.73\times$ static caching at $O=0$ and $1.54\times$ at $O=8$.

5.8 Live billing validation

We replayed ten three-task sessions ($N=100$) for four strategies against `gpt-5-nano`: 1,151 requests, 15.0M billed input tokens (13.2M served from cache), \$0.19. All live experiments in this paper, including the selection test and one discarded batch, cost \$0.58.

Tool-count limit (E0). Requests carrying 129 and all 525 tool definitions were both accepted. The 128-tool ceiling reported by users of other OpenAI endpoints does not apply to the Responses API on this model, so catalog size is limited by context and cost rather than by a hard cap.

Predicted vs. billed (H2). Table 5 compares the replay’s prediction with the provider’s own `cached_tokens`. The mean absolute percentage error on per-sequence input cost is 17.4%, inside the 20% we pre-registered, so H2 is supported. Two systematic deviations matter. First, billed cache hit rates run 3–7 points below prediction, because OpenAI caches in 128-token increments above a 1,024-token minimum and its cache is best-effort, while the replay assumes an exact-prefix, always-available cache. Second, and more importantly, the advantage of every non-static strategy

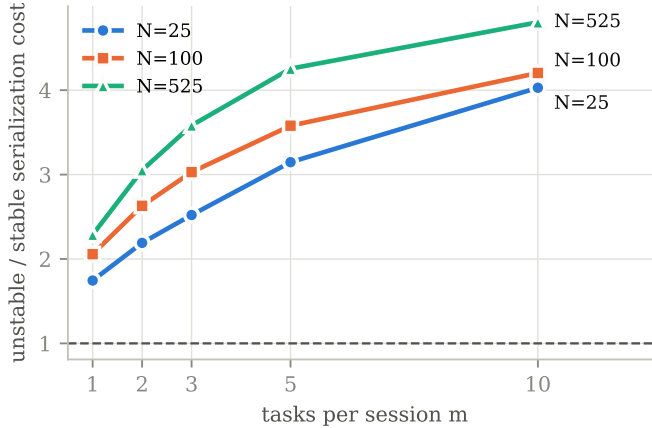


Figure 4: Cost of static caching with non-deterministic schema serialization ($p=0.44$ per call) relative to stable serialization, P1.

Table 5: Live validation on **gpt-5-nano** (10 sessions $\times$ 3 tasks, $N=100$, 1,151 requests). Cost ratios are per session against S1.

Strategy	Cache hit rate		Cost vs. S1	
	billed	predicted	billed	predicted
S1 static, cached	89%	94%	1.00	1.00
S2r RAG-prefix, per request	76%	83%	0.70	0.53
S3 summaries + tail schemas	81%	85%	0.72	0.51
S4 discovery, append-only	88%	94%	1.08	0.66

is smaller in billing than in replay: S2r costs $0.70\times$ static caching rather than the predicted $0.53\times$, and S4 costs $1.08\times$ rather than $0.66\times$. The replay therefore overstates what retrieval and discovery save, which means the cost crossover of Section 5 arrives earlier in real billing than our replay places it. Our central claim is conservative under this bias.

Practitioner pitfalls found while measuring. Two mechanical issues cost us a full measurement run and are worth reporting. (i) Several real MCP servers publish JSON-Schema draft-04 artefacts (a boolean `exclusiveMinimum`) that the API rejects outright; a client that forwards MCP schemas unmodified will fail on them. (ii) When `max_output_tokens` is small enough that the model exhausts it during reasoning, the response returns `incomplete` with all usage fields zero, so a naive measurement harness silently records no tokens. Anyone measuring cache behavior on reasoning models should assert that usage is non-zero.

Selection accuracy (H4). For each of 93 tasks we asked the model to choose the tool the recorded agent actually executed, among N candidates drawn from the same catalog. Accuracy falls monotonically as the catalog grows: 71.0% at $N=16$ (95% Wilson CI 61.1–79.2), 61.3% at 32, 54.3% at 64 and 40.9% at 128 (31.4–51.0). On the 93 tasks answered at both sizes, moving from 16 to 128 candidates flips 30 tasks from correct to incorrect and only 2 in the other direction (McNemar $\chi^2 = 22.8$, $p < 0.0001$). H4 is supported: exposing a large catalog costs selection accuracy, which is the counterweight to the cost advantage static caching gains in long sessions.

A reproducibility caveat. An earlier collection of the same 372 items, run in several resumed batches over four days, produced an anomalous 8.6% at $N=16$ in which the model selected the first listed tool in 93 of 93 cases, while agreeing with the reported run at $N \geq 32$ (58.1%, 53.8%, 35.5%). We could not reproduce that degenerate behavior in fresh probes of the same items and report the single-batch run instead, with the anomaly recorded here. Model aliases can change under a long-running experiment; measurements of this kind should be collected in one batch and the exact model snapshot recorded.

5.9 Robustness of the central claim

Three assumptions carry most of the weight in the results above. We test each.

Sessions overlap, so the intervals above are too narrow. Each task begins one session and reappears in the $m-1$ sessions that follow it, so the paired bootstrap treats correlated sessions as independent. We therefore rebuild the grid from disjoint sessions, in which no task is reused (9 sessions at $m=10$, 31 at $m=3$). The point estimates barely move and the intervals widen as expected: per-step retrieval at $m=10$, $N=100$ is $1.60\times$ with a cluster-valid interval of $[1.45, 1.74]$ (against $[1.56, 1.65]$ from overlapping sessions), and at $N=25$ it is $1.84\times$ $[1.71, 1.99]$. Tail placement is $0.82\times$ $[0.81, 0.84]$. Every interval that matters still excludes 1, so the inversion survives valid inference; we treat the disjoint intervals as the load-bearing ones.

The per-request retriever was an oracle. S2r’s tool set is the union of what the recorded router actually returned, which no deployed retriever knows in advance. Replacing it with a real BM25 retriever over tool names, descriptions and parameter names, queried with the user’s request, leaves cost essentially unchanged ($1.30\times$ vs. $1.31\times$ at $m=10$, $N=100$, since set size is what drives cost) but collapses quality: the BM25 top-5 contains the tool the agent actually executed in only 18.3% of executions, against 64.8% for the trace’s embedding router and 99.2% for the oracle. The retrieval cost estimates in this paper are therefore generous to retrieval: they price a retriever far better than a simple lexical one.

Holding agent behavior fixed favours static exposure. Section 5.8 shows selection accuracy falling with catalog size, so a static-exposure agent should make more wrong calls and pay for them. We add that cost: interpolating the measured accuracy to each strategy’s visible schema count and charging one extra call per wrong selection. The effect runs against our headline rather than for it, because static exposure is the strategy penalised most and it is the denominator. At $m=10$, $N=100$, per-step retrieval moves from $1.62\times$ to $1.38\times$; at $N=200$ it moves from $1.23\times$ to $0.97\times$. The inversion therefore holds at moderate catalogs, but its boundary moves inward once selection errors are priced, and at $N=200$ it disappears. We report this as a limit on the claim, not as support for it.

5.10 What the billed data do and do not confirm

The replay ranks tail placement (S3) ahead of per-request retrieval (S2r) everywhere, but the single cell we could bill puts them level: $0.72\times$ and $0.70\times$ at $m=3$, $N=100$ (Table 5), where the replay predicted $0.51\times$ and $0.53\times$. Two conclusions follow. First, the ordering of the cheap strategies is not established by billing; our placement recommendation rests on the replay grid and on the mechanism, not on a measured win. Second, the replay’s error is not uniform across strategies:

append-only discovery was predicted at $0.66\times$ and billed at $1.08\times$, a 64% relative error on precisely the design both major providers ship. Readers should treat the replay as reliable for the direction and magnitude of the static-versus-retrieval comparison, which the billing reproduces, and as unreliable for fine distinctions among the cache-friendly designs.

6 Discussion

Guidance for MCP client and gateway builders.

1. Serialize tool definitions deterministically. Canonical key order and stable tool order cost nothing, and non-determinism erased most of caching’s benefit in our replay.
2. Do not put a changing tool set at the top of the prompt in a long conversation. If tools must change, place their schemas after the history (S3) or append them (S4, provider tool search).
3. Do the break-even arithmetic. Per-request retrieval into the prefix pays only if the omitted catalog exceeds about $\frac{W-R}{2R}\bar{h}(m-1)$ tokens, roughly 5.75 turns’ worth of history growth per re-retrieval under 1.25/0.1 pricing.
4. Growing the tool set monotonically is not a fix. Every addition is a prefix change.
5. For short, single-request agents, retrieval is fine. The token-reduction results in the literature hold there.
6. Loading everything is cheap, not free. Selection accuracy fell from 71% to 41% as the visible catalog grew from 16 to 128 tools (Section 5.8). Static caching’s cost advantage in long sessions is paid for in accuracy, which is exactly why placement-aware designs are attractive: they keep the model’s visible tool set small and keep the prefix stable.

A decision procedure. Measure $\bar{h}$ (tokens your turns add) and m (requests per conversation); compute the threshold $\frac{W-R}{2R}\bar{h}(m-1)$; compare it with the token size of the catalog you would omit. Above the threshold, retrieve; below it, load everything once and keep the prefix stable; in either case put schemas that change after the history. For the workload measured here ($\bar{h} = 949$, six calls per request), the threshold is roughly 5.5k tokens per additional request under 1.25/0.1 pricing, i.e. about 24 average tools.

What this is worth in money. At the verified **gpt-5-nano** rate (\$0.05/M input, \$0.005/M cached), 1,000 ten-task sessions over a 100-tool catalog cost \$27.65 with static caching, \$44.73 with per-step retrieval and \$22.68 with tail placement: a \$17 penalty per thousand sessions for the token-minimizing choice, and \$5 of savings for the placement-aware one. Scale linearly for frontier-model rates.

Two stakeholders beyond client authors. MCP server authors set the catalog term directly: the median tool in our catalog costs 137 tokens and the largest costs 2,044, so trimming the tail moves the break-even more cheaply than any client-side change. Providers could remove the serialization hazard outright by canonicalizing tool definitions before hashing the prefix, and could make the trade-off visible by reporting tool-block tokens separately in usage.

Implications for research. Tool-retrieval papers should report cost under at least one cache-aware billing model, over multi-request sessions, alongside raw tokens. Raw-token reductions measured on single queries overstate savings for conversational deployments and can reverse their sign. Benchmarks that already release full trajectories, as LiveMCPBench does, make this analysis free.

Implications for the protocol. MCP permits tool-list changes and delivers `notifications/tools/list_changed` to subscribed clients so they can refresh the list [2]. The specification already recommends deterministic tool order for cache reuse; clients and gateways should serialize unchanged definitions just as consistently, which our Section 5.5 result prices. One caveat: genuine removals and authorization changes must be reflected in the model-visible tool set. Append-only exposure is a placement strategy for newly discovered tools, not a rule for every list change.

7 Threats to Validity

Internal. Replay holds agent behavior fixed across strategies, whereas in reality the tools the model sees change what it does, including how many calls it makes. We partly address this with the availability proxy and the pending live accuracy test, but cost differences below roughly 10% should not be over-interpreted. Caching is modeled as deterministic; real caches are best-effort and quantized to 128-token increments, which we do not model. Section 5.8 measures the resulting bias: it favours the non-static strategies, so our crossover estimates are conservative.

Construct. We use one tokenizer (`o200k_base`) as a proxy for all providers and an output-to-input price ratio of 5. The conclusions do not depend on that ratio (Section 5). List prices differ from negotiated prices, so we report ratios and publish token counts by type for repricing.

External. We lack a prevalence estimate for per-request retrieval into a leading tool block. A non-random source and documentation spot check of Cline, Continue, Gemini CLI and OpenCode found registered or mode-filtered tool exposure in the inspected paths, and no documented per-request prefix-resident retriever [5, 6, 7, 8]; this does not establish how often any pattern occurs across clients, providers, users or requests. The strategy ordering among cache-friendly designs is not confirmed by billing (Section 5.10), and our retrieval costs assume a retriever much stronger than BM25 (Section 5.9). All trajectories come from one agent (Claude Sonnet 4 with an embedding router) on one benchmark. Multi-task sessions are synthesized by chaining independent tasks, which lacks the cross-task context reuse of real conversations. S2r’s retrieved sets come from the agent’s own discovery calls and are optimistic.

Conclusion validity. H1, H2 and H4 are supported, H1 under cluster-valid intervals from disjoint sessions. H1’s window ($N \leq 150$, $T \geq 20$) describes a regime, not the ecosystem, and long sessions are synthesized rather than observed. H3 is supported directly on the cost side; on the accuracy side it rests on an inference (S3 and S4 expose about five schemas at a time, close to our $N=16$ condition, whereas static exposure at $N \geq 100$ sits near our 128-candidate condition), not on a measurement of those strategies end to end. The break-even rule’s agreement is with our replay, not yet with billed cost.

8 Conclusion

In the evaluated replay, token-optimal MCP tool retrieval is cost-optimal for single short tasks and cost-foolish for long multi-request sessions over moderate catalogs, when re-selecting tools invalidates an ever-larger cached history. The prevalence of that pattern in deployed clients remains to be measured. A simple rule predicts the crossover, and pricing selection errors moves it inward without removing it. Two engineering choices outweigh the choice of retriever: where retrieved schemas are placed, and whether tool definitions serialize deterministically. We recommend that future tool-retrieval work report cache-aware cost over sessions, not raw tokens over queries.

Declarations

Data availability. Code (trace extraction, replay, break-even analysis, figures and live-validation harness) and derived data is available at <https://github.com/Anoshor/penny-wise-cache-foolish> (MIT license). LiveMCPBench is available under the Apache-2.0 license [16].

Ethics. This study uses only publicly released benchmark trajectories and involves no human participants or personal data.

Author contributions (CRediT). A.B.Paul: Conceptualization, Methodology, Software, Formal analysis, Investigation, Writing – original draft, Writing – review & editing. C.Pradhan: Methodology, Software. K.Maloo: Software, Writing – review & editing.

Competing interests. The author works in industry on agentic AI systems. No employer data, systems or internal findings were used, and the work was conducted independently. [PENDING: author to confirm wording]

Funding. None.

Use of AI tools. An AI assistant (Claude, Anthropic) was used for literature search, code development, analysis and drafting under the author’s direction. The author reviewed all content and takes full responsibility for it. Pricing figures from Anthropic, OpenAI and Google documentation were retrieved on 2026-09-19.

References

- [1] Anthropic, “Introducing the Model Context Protocol,” Nov. 25, 2024. [Online]. Available: <https://www.anthropic.com/news/model-context-protocol>
- [2] Model Context Protocol, “Tools,” Specification 2026-07-28. Accessed: Oct. 3, 2026. [Online]. Available: <https://modelcontextprotocol.io/specification/2026-07-28/server/tools>
- [3] L. Zheng et al., “SGLang: Efficient execution of structured language model programs,” arXiv:2312.07104, 2023.
- [4] vLLM Project, “Automatic prefix caching,” vLLM Documentation. Accessed: Oct. 3, 2026. [Online]. Available: https://docs.vllm.ai/en/latest/design/prefix_caching/

- [5] Cline, “SDK tools,” source documentation. Accessed: Oct. 3, 2026. [Online]. Available: <https://github.com/cline/cline/blob/main/docs/sdk/tools.mdx>
- [6] Continue, “How agent mode works,” source documentation. Accessed: Oct. 3, 2026. [Online]. Available: <https://github.com/continuedev/continue/blob/main/docs/ide-extensions/agent/how-it-works.mdx>
- [7] Gemini CLI, “ToolRegistry.getFunctionDeclarations,” source code. Accessed: Oct. 3, 2026. [Online]. Available: <https://github.com/google-gemini/gemini-cli/blob/main/packages/core/src/tools/tool-registry.ts>
- [8] OpenCode, “Session tool assembly,” source code. Accessed: Oct. 3, 2026. [Online]. Available: <https://github.com/anomalyco/opencode/blob/dev/packages/opencode/src/session/tools.ts>
- [9] T. Gan and Q. Sun, “RAG-MCP: Mitigating prompt bloat in LLM tool selection via retrieval-augmented generation,” arXiv:2505.03275, 2025.
- [10] X. Fei, X. Zheng, and H. Feng, “MCP-Zero: Active tool discovery for autonomous LLM agents,” arXiv:2506.01056, 2025.
- [11] S. Mudunuri, J. Wan, A. Qin, and S. Manoharan, “Semantic tool discovery for large language models: A vector-based approach to MCP tool selection,” arXiv:2603.20313, 2026.
- [12] A. Sadani and D. Kumar, “Tool attention is all you need: Dynamic tool gating and lazy schema loading for eliminating the MCP/tools tax in scalable agentic workflows,” arXiv:2604.21816, 2026.
- [13] E. Lumer, F. Nizar, A. Jangiti, K. Frank, A. Gulati, M. Phadate, and V. K. Subbiah, “Don’t break the cache: An evaluation of prompt caching for long-horizon agentic tasks,” arXiv:2601.06007, 2026.
- [14] B. Xu et al., “TokenPilot: Cache-efficient context management for LLM agents,” arXiv:2606.17016, 2026.
- [15] Y. Fang, S. Wei, H. Hu, and X. Shen, “ReCache: Efficient KV cache reuse and compression for tool-augmented LLM agents,” arXiv:2608.19662, 2026.
- [16] G. Mo, W. Zhong, J. Chen, Q. Yuan, X. Chen, Y. Lu, H. Lin, B. He, X. Han, and L. Sun, “LiveMCP-Bench: Can agents navigate an ocean of MCP tools?” arXiv:2508.01780, 2025.
- [17] Z. Wang et al., “MCP-Bench: Benchmarking tool-using LLM agents with complex real-world tasks via MCP servers,” arXiv:2508.20453, 2025.
- [18] Z. Luo et al., “MCP-Universe: Benchmarking large language models with real-world Model Context Protocol servers,” arXiv:2508.14704, 2025.
- [19] Z. Guo, B. Xu, C. Zhu, W. Hong, X. Wang, and Z. Mao, “MCP-AgentBench: Evaluating real-world language agent performance with MCP-mediated tools,” arXiv:2509.09734, 2025.
- [20] M. M. Hasan, H. Li, G. K. Rajbahadur, B. Adams, and A. E. Hassan, “Model Context Protocol (MCP) tool descriptions are smelly! Towards improving AI agent efficiency with augmented MCP tool descriptions,” arXiv:2602.14878, 2026.
- [21] Y. Felendler, P. A. Gandhi, I. Habler, Y. Elovici, and A. Shabtai, “From tool orchestration to code execution: A study of MCP design choices,” arXiv:2602.15945, 2026.
- [22] Anthropic, “Prompt caching,” Claude Platform Docs. Accessed: Sep. 19, 2026. [Online]. Available: <https://platform.claude.com/docs/en/build-with-claude/prompt-caching>
- [23] Anthropic, “Tool search tool,” Claude Platform Docs. Accessed: Sep. 19, 2026. [Online]. Available: <https://platform.claude.com/docs/en/agents-and-tools/tool-use/tool-search-tool>

- [24] OpenAI, “Prompt caching,” OpenAI API Docs. Accessed: Sep. 19, 2026. [Online]. Available: <https://developers.openai.com/api/docs/guides/prompt-caching>
- [25] OpenAI, “Tool search,” OpenAI API Docs. Accessed: Sep. 19, 2026. [Online]. Available: <https://developers.openai.com/api/docs/guides/tools-tool-search>
- [26] OpenAI, “Pricing,” OpenAI API Docs. Accessed: Sep. 19, 2026. [Online]. Available: <https://developers.openai.com/api/docs/pricing>
- [27] Google, “Gemini Developer API pricing.” Accessed: Sep. 19, 2026. [Online]. Available: <https://ai.google.dev/gemini-api/docs/pricing>
- [28] maximhq/bifrost, “MCP tool schema property order changes between tool syncs, breaking prompt caching,” GitHub issue #7169. [Online]. Available: <https://github.com/maximhq/bifrost/issues/7169>
- [29] code-yeongyu/oh-my-openagent, “Too many tools registered (141) exceeds OpenAI API limit of 128,” GitHub issue #2848. [Online]. Available: <https://github.com/code-yeongyu/oh-my-openagent/issues/2848>